Les Structures Répétitives en Algorithmique et Langage C : Boucles While, Do-While et For

Thème principal : Programmation Impérative et Structures Itératives

Synthèse exécutive : Ce cours magistral aborde en profondeur les structures répétitives (boucles) en algorithmique et en langage C. L'enjeu fondamental est d'automatiser l'exécution d'un bloc d'instructions à répétition sans recourir à la duplication de code, en respectant scrupuleusement le principe DRY (Don't Repeat Yourself). Sont étudiées la boucle 'while' (pré-condition, de 0 à N itérations, idéale pour les événements dynamiques), la boucle 'do-while' (post-condition, 1 à N itérations garanties, indispensable pour les contrôles de saisie) et la boucle 'for' (itérations déterminées, centralisant initialisation, condition et incrémentation). Une attention particulière est portée sur la prévention des boucles infinies, la gestion des variables de contrôle et les pièges syntaxiques fréquents en langage C (comme le point-virgule obligatoire en fin de do-while).

Utilité des structures itératives et principe DRY

En programmation, la répétition de tâches ne doit jamais être gérée par la copie manuelle de lignes de code. Si un algorithme doit traiter 10 000 éléments d'un tableau ou redemander une saisie jusqu'à ce qu'elle soit correcte, dupliquer le code rend le programme lourd, illisible et impossible à maintenir. Les structures itératives permettent d'automatiser cette répétition en exécutant un bloc d'instructions tant qu'une condition logique demeure vraie. L'erreur critique à éviter est la boucle infinie, causée par l'oubli de la mise à jour de la variable de contrôle à l'intérieur du bloc.

- Principe DRY (Don't Repeat Yourself) : interdiction absolue de dupliquer du code par copier-coller pour maintenir la lisibilité et éviter les bogues.
- Condition d'arrêt : expression booléenne évaluée pour déterminer la poursuite ou l'interruption de la boucle.
- Variable de contrôle / Compteur : variable modifiée au fil des itérations pour faire progresser la condition vers sa valeur d'arrêt.
- Boucle infinie : dysfonctionnement critique bloquant le programme lorsque la condition d'arrêt ne devient jamais fausse.

Cas concret enseignant : *Métaphore du tapis roulant d'usine d'emballage : tant qu'un colis est détecté par le capteur (condition vraie), le bras robotisé l'attrape et l'emballe. Si le capteur ne détecte plus de colis ou si le quota est atteint, le robot s'arrête. Sans condition de fin fonctionnelle, le robot continuerait à tourner à vide jusqu'à surchauffer et détruire la machine.*

La boucle TANT QUE (while) : Structure à pré-condition

La structure 'while' est la boucle de référence lorsque le nombre d'itérations n'est pas connu à l'avance, car il dépend d'un événement dynamique externe (saisie utilisateur, fin de lecture d'un fichier, signal réseau). Comme le test est effectué en amont, si la condition est fausse dès le départ, le code interne est totalement ignoré. Il est impératif d'assurer l'initialisation des variables avant la boucle et leur modification explicite à l'intérieur pour garantir l'arrêt du programme.

- Pré-conditionnement : la condition booléenne est évaluée AVANT chaque exécution du bloc d'instructions.
- Intervalle d'itérations : de 0 à N passages (le bloc peut ne jamais s'exécuter si la condition est fausse dès l'entrée).
- Initialisation préalable : toutes les variables testées dans la condition doivent obligatoirement être initialisées avant l'instruction while.

Cas concret enseignant : *Exemple du contrôle du code PIN au distributeur de billets : la machine demande le code TANT QUE le code saisi est incorrect ET QUE le nombre d'essais est inférieur à 3. Si l'utilisateur tape le bon code du premier coup, la boucle s'arrête immédiatement sans répéter la demande.*

La boucle FAIRE... TANT QUE (do-while) : Structure à post-condition

À la différence du 'while', la boucle 'do-while' garantit que les instructions de son corps sont exécutées une première fois avant qu'aucun test ne soit réalisé. Elle est idéalement conçue pour le contrôle de saisie (validation de bornes numériques) et les menus interactifs. En langage C, l'oubli du point-virgule final de l'instruction while provoque systématiquement une erreur de compilation.

- Post-conditionnement : le test logique est réalisé APRÈS l'exécution du bloc d'instructions.
- Garantie d'exécution : la boucle s'exécute obligatoirement au minimum 1 fois (intervalle de 1 à N itérations).
- Syntaxe C stricte : obligation impérative d'apposer un point-virgule ';' après la parenthèse fermante du while : do { ... } while(condition);

Cas concret enseignant : *Cas du contrôle de saisie d'une note d'examen : 'Il est techniquement impossible de vérifier si la saisie faite par un étudiant est valide avant qu'il n'ait tapé au moins une valeur sur le clavier. On doit donc exécuter la saisie au moins une fois, ce qui fait du do-while la structure parfaitement adaptée.'*

La boucle POUR (for) : Itération déterminée

La structure 'for' rationalise la mécanique itérative en centralisant sur une seule ligne les trois étapes vitales de l'itération. Bien que les trois expressions séparées par des points-virgules soient techniquement optionnelles en langage C, les omettre nuit gravement à la lisibilité et constitue une mauvaise pratique fortement sanctionnée en évaluation.

- En-tête condensé : regroupe l'initialisation, la condition de maintien et l'incrémentation sous la forme for(init; cond; inc).
- Usage privilégié : réservée aux situations où le nombre exact d'itérations est connu avant d'entrer dans la boucle (ex: parcours de tableaux).
- Portée (scope) : possibilité de déclarer la variable de contrôle directement au sein de l'en-tête (ex: for(int i = 0; ...)).

Cas concret enseignant : *Métaphore de la course sur piste d'athlétisme : 'Vous savez avant de vous lancer que vous devez effectuer exactement 5 tours. Vous initialisez votre compteur au tour 1, vous vérifiez que vous n'avez pas dépassé 5, et à chaque franchissement de la ligne d'arrivée, vous ajoutez 1 à votre compteur.'*

■ Questions & Interventions des Étudiants en Classe

Question étudiant : Pourquoi le programme plante ou bloque indéfiniment quand j'oublie d'incrémenter 'i' dans une boucle while ?
Réponse du professeur : Parce que la condition d'arrêt repose sur une variable (comme 'i') qui doit évoluer. Si 'i' ne change pas, la condition reste éternellement vraie, ce qui provoque une boucle infinie qui gèle le processus et consomme 100% des ressources du processeur.

Question étudiant : Est-ce qu'on peut toujours remplacer une boucle 'for' par une boucle 'while' ?
Réponse du professeur : Oui, absolument. Toute boucle 'for' peut s'écrire sous forme de boucle 'while' en initialisant le compteur avant, en plaçant la condition dans le while, et en incrémentant à la fin du bloc. Le 'for' n'est qu'un sucre syntactique qui améliore la lisibilité quand le nombre d'itérations est connu.

■ Exercice & Démonstration d'Application

Énoncé : Écrire un programme complet en langage C qui calcule la somme des N premiers entiers positifs (ex: si N = 5, calculer 1 + 2 + 3 + 4 + 5 = 15). Le programme doit obligatoirement valider que N est un entier strictement positif (> 0) à l'aide d'une boucle 'do-while', puis calculer la somme au moyen d'une boucle 'for'.

Résolution pas à pas :
Étape 1 : Déclarer les variables entières n, somme (initialisée à 0) et i. Étape 2 : Mettre en place la saisie sécurisée avec la structure do-while : do { printf("Entrez N (>0) : "); scanf("%d", &n;); } while (n <= 0); Étape 3 : S'assurer que somme = 0 avant le calcul. Étape 4 : Écrire la boucle for pour effectuer le cumul de 1 jusqu'à N : for (i = 1; i <= n; i++) { somme += i; } Étape 5 : Afficher le résultat avec printf("La somme des %d premiers entiers est %d\n", n,

somme);

Résultat : Pour une saisie où l'utilisateur entre d'abord -2 puis 5 : La boucle do-while rejette -2 et réclame une nouvelle saisie. Lorsque 5 est entré, la boucle do-while valide la valeur. La boucle for réalise 5 itérations (i=1 -> somme=1 ; i=2 -> somme=3 ; i=3 -> somme=6 ; i=4 -> somme=10 ; i=5 -> somme=15). Le programme affiche correctement : 'La somme des 5 premiers entiers est 15'.

■ Auto-Évaluation Flash d'Examen

Q : Quelle est la différence fondamentale d'évaluation entre une boucle 'while' et une boucle 'do-while' ?

R : Le 'while' évalue sa condition en amont (pré-condition, 0 à N itérations, le bloc peut ne jamais s'exécuter), tandis que le 'do-while' évalue sa condition en aval après un premier passage obligatoire (post-condition, 1 à N itérations garanties).

Q : Que se passe-t-il si la condition initiale d'une boucle 'for' est fausse dès la première évaluation ?

R : Le corps de la boucle n'est exécuté aucune fois (0 itération) et le programme poursuit son cours directement après la boucle.

Q : Quel oubli de syntaxe systématique provoque une erreur de compilation sur la boucle 'do-while' en langage C ?

R : L'oubli du point-virgule ';' final après la parenthèse fermante de l'instruction while (ex: do { ... } while(condition);).